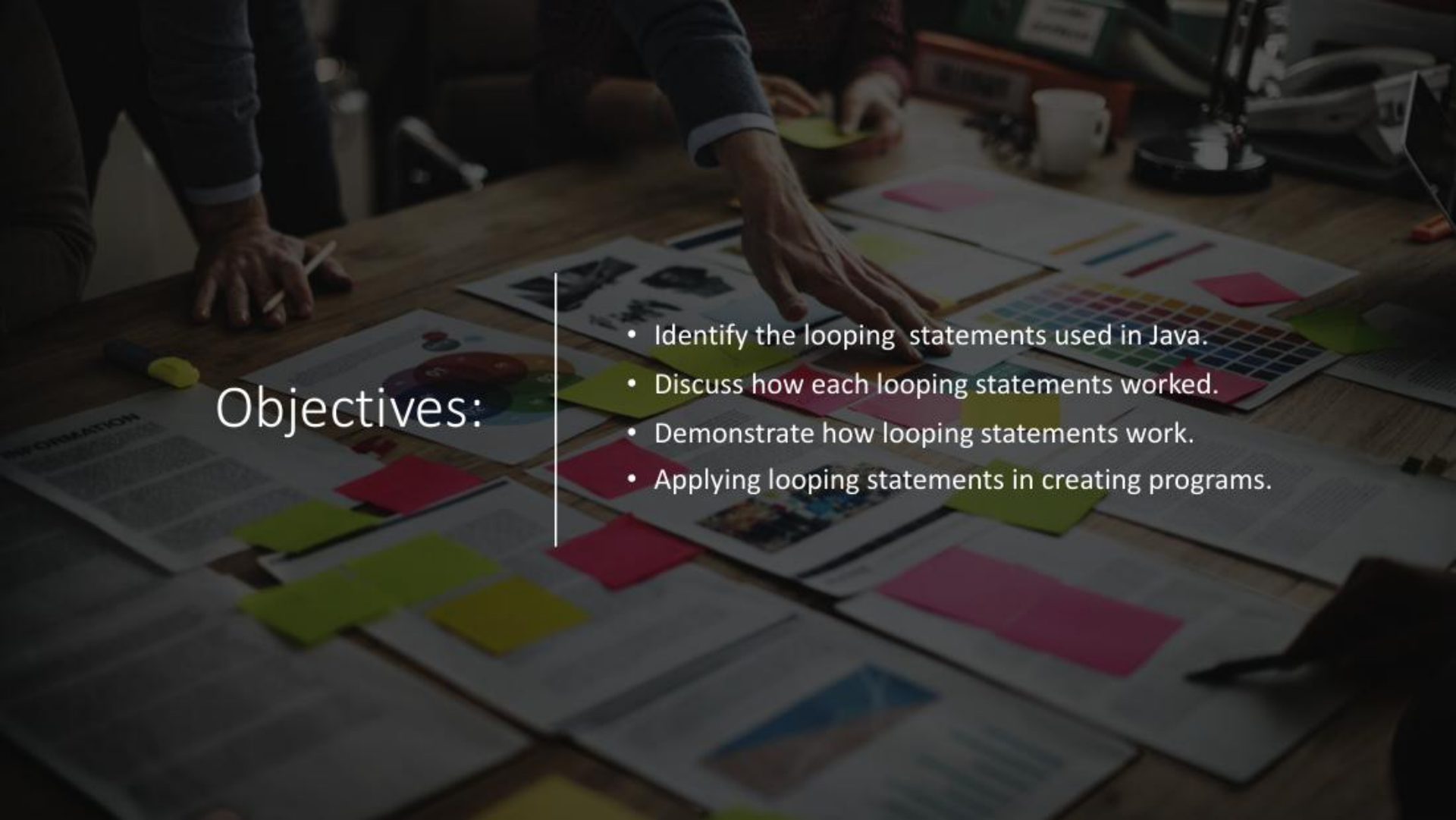
Looping Statements in Java



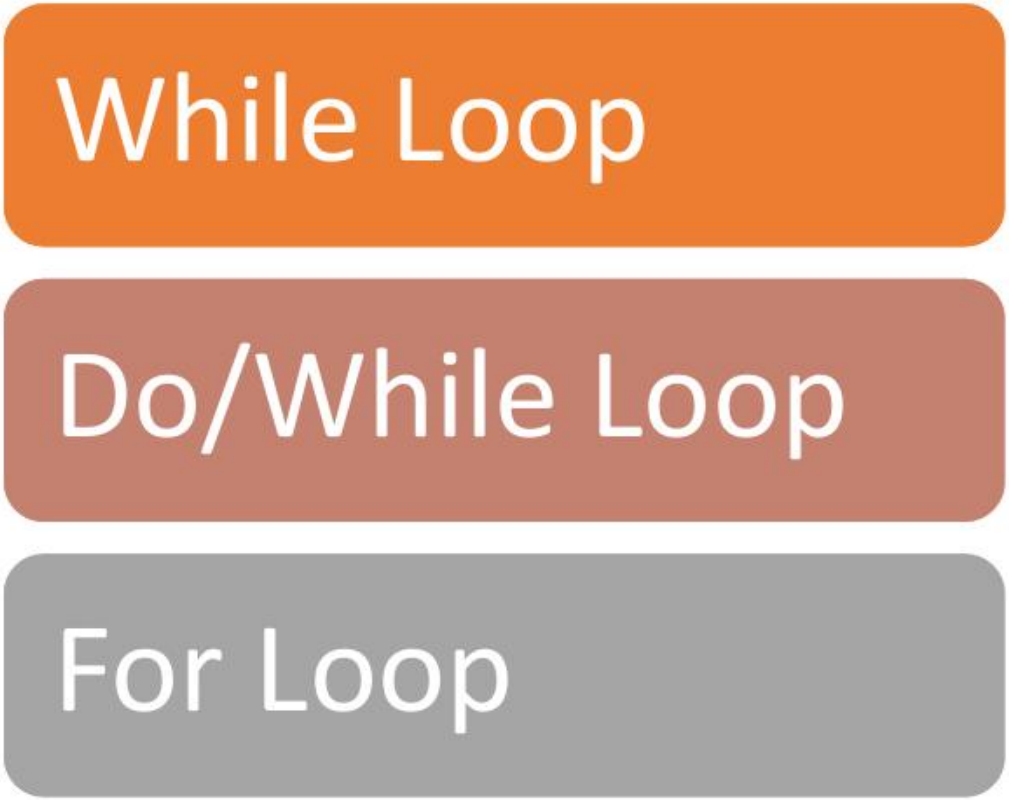
Objectives:

- Identify the looping statements used in Java.
- Discuss how each looping statements worked.
- Demonstrate how looping statements work.
- Applying looping statements in creating programs.

What is a looping statement?

- A loop statement is a series of steps or sequence of statements executed repeatedly zero or more times satisfying the given condition is satisfied.
- Loops can execute a block of code as long as a specified condition is reached.
- Loops are handy because they save time, reduce errors, and they make code more readable.

Loops in Java



Java While Loop

- The ***while loop*** loops through a block of code as long as a specified condition is true:

SYNTAX:

```
while (condition) {
  // code block to be executed
}
```

Example

In the example below, the code in the loop will run, over and over again, as long as a variable (i) is less than 5:

```
int i = 0;
while (i < 5) {
System.out.println(i);
 i++;
}
```

The Do/While Loop

- The ***do/while loop*** is a variant of the while loop.
- This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

SYNTAX:

```
do {
  // code block to be executed
}
while (condition);
```

EXAMPLE

- The example below uses a do/while loop. The loop will always be executed at least once, even if the condition is false, because the code block is executed before the condition is tested:

```
int i = 0;
do {
 System.out.println(i);
i++;
}
while (i < 5);
```

Example 2:

- Create a program using do/while that displays the even numbers from 1 to 10.

Sample code:

```
int i=2;
    do {
        System.out.println(i);
        i=i+2;
    }
    while (i<=10);
```

Sample Problems

1. Create a program that displays the even numbers from 1 to 10.
2. Create a program that displays numbers divisible by 5 from 1-50.
3. Create a program that displays odd numbers from 1 to 20.

Assignment:

1. Create a program using while or do/while loop that accepts input from the user once. If the user inputs an odd number, it will display odd numbers from 1 to 10. If the user inputs an even number, even numbers from 1 to 10 will be displayed. If the user inputs 0 or any negative number, it will display invalid input.

Sample Program Output 1:

```
Enter a number: 1
Odd numbers from 1 to 10 are:
1
3
5
7
9
```

Sample Program Output 2:

```
Enter a number: 2
Even numbers from 1 to 10 are:
2
4
6
8
10
```

Sample Program Output 3:

```
Enter a number: 0
Invalid input
```

Java For Loop

- When you know exactly how many times you want to loop through a block of code, use the for loop instead of a while loop:
- SYNTAX:

```
for (statement 1; statement 2; statement 3) {
  // code block to be executed
}
```

SYNTAX EXPLAINED

- **Statement 1** is executed (one time) before the execution of the code block.
- **Statement 2** defines the condition for executing the code block.
- **Statement 3** is executed (every time) after the code block has been executed.
-

EXAMPLE:

```
for (int i = 0; i < 5; i++) {
  System.out.println(i);
}
```

- Statement 1 sets a variable before the loop starts (int i = 0).
- Statement 2 defines the condition for the loop to run (i must be less than 5). If the condition is true, the loop will start over again, if it is false, the loop will end.
- Statement 3 increases a value (i++) each time the code block in the loop has been executed.

This example will only print even values between 0 and 10:

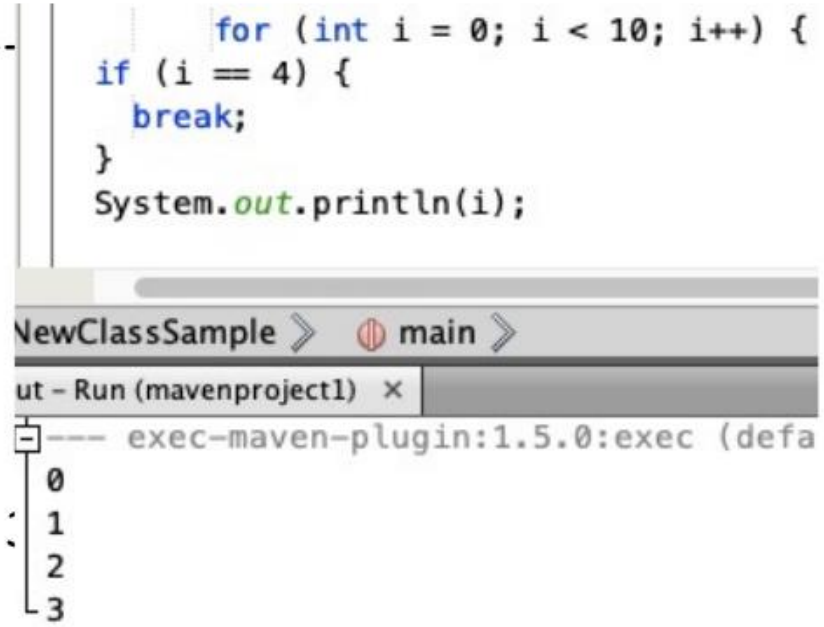
```
for (int i = 0; i <= 10; i = i + 2) {
  System.out.println(i);
}
```

Java Break and Continue

- Break is used to "jump out" of a switch statement.
- The break statement can also be used to jump out of a **loop**.

This example jumps out of the loop when i is equal to 4:

```
for (int i = 0; i < 10; i++
  if (i == 4) {
    break;
  }
  System.out.println(i);
}
Expected output: 0, 1, 2, 
```



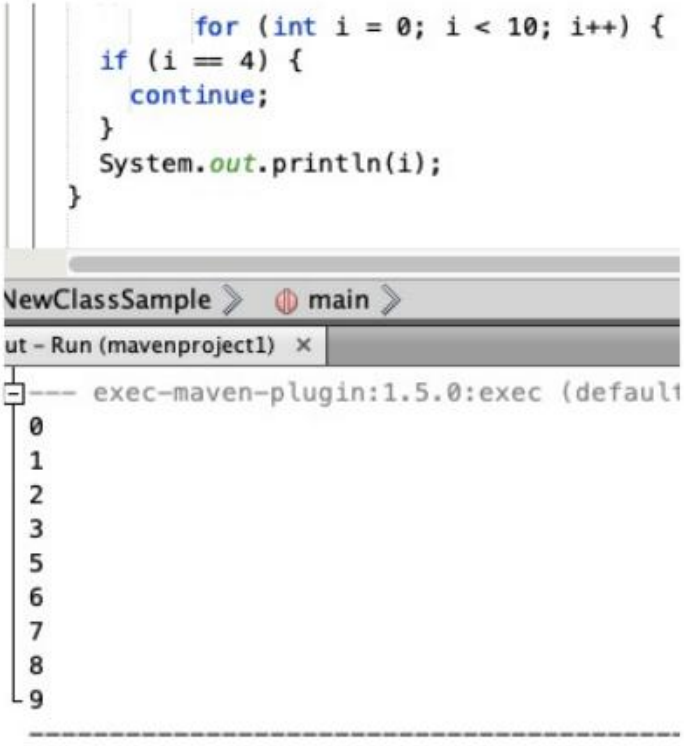
Java Continue

- The continue statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

Example:

- This example skips the value of 4:

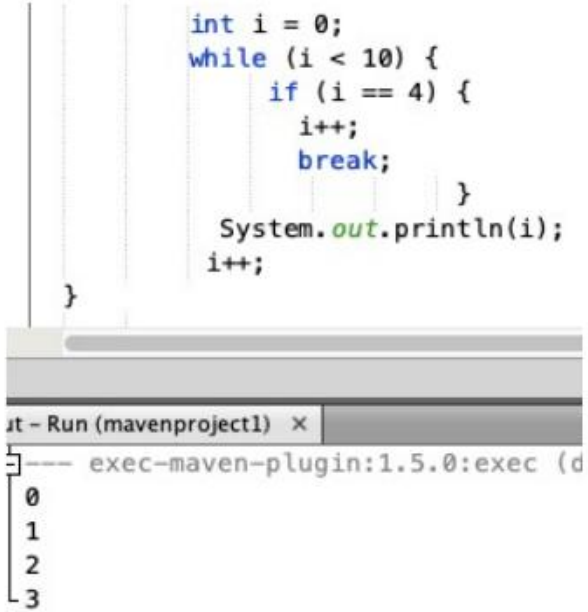
```
for (int i = 0; i < 10; i++) {
   if (i == 4) {
     continue;
   }
   System.out.println(i);
}
```



Break and Continue in While Loop

- You can also use break and continue in while loops:
- Break Example

```
int i = 0;
while (i < 10) {
  System.out.println(i)
  i++;
  if (i == 4) {
    break;
  }
}
```



Continue Example

```
int i = 0;
while (i < 10) {
  if (i == 4) {
    i++;
    continue;
  }
  System.out.println(i);
       i++;
}
```

```
        int i = 0;
while (i < 10) {
  if (i == 4) {
    i++;
    continue;
  }
  System.out.println(i);
        i++;
}
```

t - Run (mavenproject1) ×

```
]--- exec-maven-plugin:1.5.0:exec (default-c
0
1
2
3
5
6
7
8
-9
```